
Entropy Arena

Looking Random Is Not Being Unpredictable:
A Mathematical Framework for Comparing Randomness Sources
in Cryptographic Key Generation

Entropy Arena Project

September 22, 2026

Abstract

Entropy Arena is an experimental framework that compares randomness sources — computational, mathematical, physical and manual — as suppliers of seeds and private keys for a 2-of-3 multisignature Bitcoin wallet. Its central thesis is that *statistical resemblance to randomness and unpredictability are different properties*: a deterministic process, however chaotic, inherits all of its unpredictability from the seed that starts it. We formalize this with two independent evaluation axes: a *statistical quality* Q , obtained from the NIST SP 800-22 battery aggregated by the proportion-of-passes and p -value-uniformity criteria, and a *real entropy* r , the number of unpredictable bits actually injected per sample, estimated with the min-entropy estimators of NIST SP 800-90B. We derive every metric, explain why each is used and what it cannot detect, describe the physics and dynamics of each generator and why it does or does not supply entropy, present concrete attacks that break generators that pass the whole battery (Mersenne Twister, RANDU, digits of π , clock seeds), and give the cryptographic pipeline (BIP39/32/48/67, descriptors) that turns the measured entropy into a verified regtest multisignature transaction. We give the spending policy as an indicator-function threshold rule and its equivalent disjunctive normal form, derive combinatorially why a 3-of-5 policy has $\binom{5}{3} = 10$ minimal authorized signer sets, and correct a common misconception on threshold security: in a t -of- n scheme the security is set by the t -th weakest key, not by the strongest one. We further show, under an independent per-key compromise model, that 3-of-5 is strictly safer than 2-of-3 for any realistic compromise probability ($P_{\text{break}} \sim \binom{n}{t} p^t$, and $10p^3 < 3p^2$ whenever $p < 0.3$), while also tolerating one additional lost key.

Contents

1	Introduction and Threat Model	3
2	Preliminaries: Measures of Entropy	3
2.1	Shannon, collision and min-entropy	3
2.2	Conditional min-entropy and conservation	4
3	Entropy Sources: Mathematics and Physics	4
3.1	Computational sources	4
3.2	Mathematical sources: deterministic dynamics	5
3.2.1	Brownian motion	5
3.2.2	Logistic map	5
3.2.3	Lorenz attractor	6
3.2.4	Rule 30 cellular automaton	6
3.2.5	Binary digits of π	6

3.3	Physical and manual sources	6
3.3.1	CPU timing jitter	6
3.3.2	Dice and coins	6
3.3.3	Deck shuffle	7
3.3.4	von Neumann extractor and XOR combination	7
4	Evaluation Metrics	7
4.1	NIST SP 800-22 statistical tests	7
4.1.1	Aggregation across samples	8
4.2	Min-entropy estimation (NIST SP 800-90B)	8
4.3	Structural tests	10
4.4	Combining the two axes	10
4.5	Sources of error in the measurements themselves	11
5	Attacks on Generators that Pass the Battery	11
6	Cryptographic Pipeline	12
6.1	From entropy to seed (BIP39)	12
6.2	Hierarchical derivation (BIP32)	12
6.3	Multisignature script and descriptors	13
6.4	Spending logic: the policy as a Boolean formula	13
6.5	Generalization: t -of- n and the combinatorics of the “10 favorable cases”	13
6.6	Verification chain	15
7	Security Composition in Threshold Schemes	15
7.1	Why 3-of-5 is more secure than 2-of-3	16
8	Results	17
8.1	Arena	17
8.2	Validation of the min-entropy estimators	18
8.3	Structural tests	18
8.4	Attacks	19
8.5	Wallet	19
9	Discussion and Limitations	19
A	Notation	20
B	Constants	20

1 Introduction and Threat Model

The security of a cryptographic key is the probability that an adversary guesses it. That probability depends on *what the adversary knows*, not on how the key “looks”. We fix the following threat model, used throughout.

Definition 1.1 (Adversary). The adversary $\mathcal{A}$ knows the source code and hardware of every generator, can observe any previous output of the generator (but not the private inputs of the current sample), and can test q candidates offline.

Let $\mathcal{M} = \{M_1, \dots, M_k\}$ be a set of entropy methods. A method consumes a possibly empty *fresh unpredictable input* ω (an OS seed, a physical measurement, dice rolls) and produces a byte string

$$D = M(\omega, n) \in \{0, 1\}^{8n}. \quad (1)$$

The arena assigns to each method two *independent* quantities:

- the **statistical quality** $Q(M) \in [0, 100]$: how well D resembles a uniform i.i.d. string under a battery of tests;
- the **real entropy** $r(M) \geq 0$ (bits): how much unpredictability ω injects, conditionally on Definition 1.1.

The reason for two axes is a theorem (Section 2.2): Q can be made maximal by any deterministic function, while r cannot exceed the entropy of ω . A single weighted score would blur this distinction; the first version of this project used one, and we replaced it (Section 4.4).

2 Preliminaries: Measures of Entropy

2.1 Shannon, collision and min-entropy

Let X be a discrete random variable with pmf $p(x)$ on a finite alphabet $\mathcal{X}$.

Definition 2.1 (Rényi entropies).

$$H_1(X) = - \sum_x p(x) \log_2 p(x) \quad (\text{Shannon}), \quad (2)$$

$$H_2(X) = - \log_2 \sum_x p(x)^2 \quad (\text{collision}), \quad (3)$$

$$H_\infty(X) = - \log_2 \max_x p(x) \quad (\text{min-entropy}). \quad (4)$$

Proposition 2.2 (Ordering). $H_\infty(X) \leq H_2(X) \leq H_1(X) \leq \log_2 |\mathcal{X}|$, and $H_2(X) \leq 2 H_\infty(X)$. All are equal iff X is uniform on its support.

Proof. Since $\sum_x p(x)^2 \leq \max_x p(x) \sum_x p(x) = \max_x p(x)$, we get $H_2 \geq H_\infty$; since $\sum_x p(x)^2 \geq (\max_x p(x))^2$, we get $H_2 \leq 2H_\infty$. Jensen’s inequality for the concave log gives $H_1 \geq H_2$. $\square$

Why min-entropy is the cryptographic measure. If the adversary makes q guesses, each guess succeeds with probability at most $\max_x p(x)$, so

$$\Pr[\mathcal{A} \text{ succeeds}] \leq q 2^{-H_\infty(X)}. \quad (5)$$

Shannon entropy averages over outcomes and can be large while one outcome is very likely; the adversary will simply try that outcome first.

Example 2.3. Let $X = 0$ with probability $1/2$ and uniform on 2^{100} other values otherwise. Then $H_1 \approx 51$ bits, but $H_\infty = 1$: half of the keys are guessed in one attempt.

2.2 Conditional min-entropy and conservation

Definition 2.4. For a random variable D and adversary knowledge Z ,

$$H_\infty(D | Z) = -\log_2 \mathbb{E}_z \left[\max_d \Pr[D = d | Z = z] \right]. \quad (6)$$

Theorem 2.5 (Entropy conservation). *Let $D = g(S)$ where g is a deterministic function known to the adversary and S is the random seed. Then $H_\infty(D | g) \leq H_\infty(S)$. Consequently a deterministic generator with an s -bit seed can never contribute more than s bits, regardless of its statistical properties.*

Proof. Let s^* be a most likely seed. Then $\Pr[D = g(s^*)] \geq \Pr[S = s^*] = \max_s \Pr[S = s]$, hence $\max_d \Pr[D = d] \geq 2^{-H_\infty(S)}$. $\square$

Remark 2.6. Theorem 2.5 is the data-processing inequality for min-entropy. It explains why applying SHA-256 to a low-entropy input gives an output that is statistically perfect (Shannon entropy ≈ 8 bits/byte, all tests passed) but has $H_\infty \leq H_\infty(\text{input})$: hashing *hides* structure, it does not *create* uncertainty.

Definition 2.7 (Real entropy r). For a method, $r(M)$ is the average, over samples, of $\min\{b(M), 8n\}$, where $b(M)$ is the number of bits of fresh unpredictable input consumed to produce the sample, conditional on Definition 1.1 (that is, given the algorithm and all previous outputs).

Examples of b : a CSPRNG gives $8n$; a mathematical map seeded with 8 OS bytes gives 64; the Mersenne Twister that persists between samples gives 0, because its state is recoverable from earlier outputs (Section 5); a manual source gives the measured min-entropy of the recorded rolls.

3 Entropy Sources: Mathematics and Physics

Methods fall into four families: *computational*, *mathematical*, *physical* and *manual/hybrid*.

3.1 Computational sources

CSPRNG (`secrets`, `os.urandom`). The operating system collects physical noise (interrupt timing, hardware RNG instructions, device jitter) into an entropy pool and expands it with a cryptographic generator $G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^n$. If G is a secure PRG,

$$|\Pr[\mathcal{A}(G(U_\lambda)) = 1] - \Pr[\mathcal{A}(U_n) = 1]| \leq \varepsilon(\lambda) \quad (7)$$

for every polynomial-time $\mathcal{A}$. Its power comes from a *fresh secret seed* of $\lambda \geq 128$ bits, not from the shape of the output.

Mersenne Twister (MT19937). The state is $\mathbf{x} \in (\mathbb{F}_2)^{19937}$ (624 words of 32 bits, less 31 unused bits), and one step is a *linear* map over $\mathbb{F}_2$:

$$x_{k+624} = x_{k+397} \oplus ((x_k^u \parallel x_{k+1}^l)A), \quad (8)$$

followed by the tempering $y \mapsto y \oplus (y \gg 11)$, $y \oplus ((y \ll 7) \wedge 0x9D2C5680)$, $y \oplus ((y \ll 15) \wedge 0xEFC60000)$, $y \oplus (y \gg 18)$, also linear over $\mathbb{F}_2$ and invertible. The period $2^{19937} - 1$ and 623-dim equidistribution [3] explain why it passes statistical batteries; the linearity explains why it is predictable (Section 5).

RANDU. The linear congruential generator $x_{k+1} = 65539 x_k \bmod 2^{31}$. Since $65539 = 2^{16} + 3$, $65539^2 \equiv 6 \cdot 65539 - 9 \pmod{2^{31}}$, hence

$$x_{k+2} = 6x_{k+1} - 9x_k \pmod{2^{31}}. \quad (9)$$

Any three consecutive outputs satisfy $9x_k - 6x_{k+1} + x_{k+2} \equiv 0$, so the points (x_k, x_{k+1}, x_{k+2}) lie on at most 15 planes [4]. The generator keeps a 31-bit state (30 bits for an odd seed), so at most 30 bits of entropy exist whatever the output looks like.

PCG64 (`numpy.random.default_rng`). A statistical (not cryptographic) generator seeded from a 128-bit **SeedSequence** drawn from the OS. Fresh seeding per sample gives $b = 128$; the generator itself is invertible, so seed reuse across samples would give 0.

UUIDv4. 122 of the 128 bits are random; 6 encode version and variant. Hence $b = n \cdot 8 \cdot 122/128$. The constant bits are visible to statistical tests: they bias the monobit, runs, cumulative-sums and chi-squared statistics, so UUIDv4 has good entropy but poor Q (Section 8).

3.2 Mathematical sources: deterministic dynamics

Every source in this family is a map $s_{k+1} = f(s_k)$ with a seed s_0 drawn from the OS (64 bits per sample); by Theorem 2.5 $b \leq 64$ regardless of dynamics. What differs is *how much of the seed's information the output reveals*, which we analyze per system.

3.2.1 Brownian motion

Physically, a Brownian particle is driven by molecular collisions: the Langevin equation $m\dot{v} = -\gamma v + \sqrt{2\gamma k_B T} \eta(t)$ has a truly stochastic term η whose source is thermal (a quantum-statistical ensemble at temperature T). The Wiener process W_t has independent Gaussian increments $W_t - W_s \sim \mathcal{N}(0, t - s)$. *In the arena, however, the noise is simulated:*

$$\xi_k = \sqrt{\Delta t} Z_k, \quad Z_k \sim \mathcal{N}(0, 1) \text{ drawn from a PRNG}, \quad b_k = \mathbf{1}[\xi_k > 0]. \quad (10)$$

Each b_k is a deterministic function of the PRNG seed, so the physical randomness of the *model* is irrelevant: the simulation has only the seed's entropy. This is a recurring lesson: *simulating a random physical process is not measuring one.*

3.2.2 Logistic map

$x_{k+1} = rx_k(1 - x_k)$. For $r = 4$ the map is conjugate to the doubling map: with $x = \sin^2(\pi\theta)$, $\theta_{k+1} = 2\theta_k \bmod 1$. Writing $\theta_0 = 0.d_1d_2d_3\dots$ in binary,

$$\mathbf{1}[x_k > \tfrac{1}{2}] = \mathbf{1}[\theta_k \in (\tfrac{1}{4}, \tfrac{3}{4}]] = d_{k+1} \oplus d_{k+2}. \quad (11)$$

The output is a linear ($\mathbb{F}_2$) filter of the binary digits of the seed: the chaotic map *spends* one seed bit per iteration (Lyapunov exponent $\lambda = \ln 2$, i.e. Kolmogorov–Sinai entropy of 1 bit/iteration), it does not create any. In double precision the state has only 53 bits, so the computed orbit is a deterministic function of at most 53 bits of the seed and is eventually periodic; after ≈ 53 iterations the extracted bits reflect floating-point rounding, not new information.

Threshold bias. At $r = 3.99$ the invariant density is no longer symmetric about $1/2$, so the threshold bit $\mathbf{1}[x > 1/2]$ is biased. This was the reason the first implementation scored poorly ($Q \approx 24$); the current implementation extracts a low-order bit, $b_k = \lfloor 2^{30} x_k \rfloor \bmod 2$, which is insensitive to the coarse density.

3.2.3 Lorenz attractor

The flow $\dot{x} = \sigma(y - x)$, $\dot{y} = x(\rho - z) - y$, $\dot{z} = xy - \beta z$ ($\sigma = 10, \rho = 28, \beta = 8/3$) has largest Lyapunov exponent $\lambda_1 \approx 0.9056$ [9]. The Kolmogorov–Sinai entropy rate is the sum of positive exponents, $h_{KS} \approx \lambda_1 / \ln 2 \approx 1.31$ bits per unit of time. With explicit Euler steps $\Delta t = 10^{-2}$ we extract one bit per step,

$$b_k = \mathbf{1}[x_{k+1} > x_k], \quad (12)$$

but the system generates only $h_{KS}\Delta t \approx 0.013$ bits per step: the source is *oversampled by a factor of ≈ 75* . Consecutive bits are the sign of a slowly varying derivative and are strongly correlated (runs of equal bits), which is why Lorenz fails four tests and has min-entropy ≈ 1.2 bits/byte. The physical lesson: a chaotic system produces information at a finite rate; sampling faster than that rate reads the same information repeatedly.

3.2.4 Rule 30 cellular automaton

$c_i^{(t+1)} = c_{i-1}^{(t)} \oplus (c_i^{(t)} \vee c_{i+1}^{(t)})$. Wolfram proposed the central column as a generator [10]: bits are extracted from $c_{\lfloor L/2 \rfloor}^{(t)}$, $t > K$. The rule is nonlinear (the $\vee$), which is why it resists linear-complexity attacks, and the column passes the tests, but the initial ring (of $L = 256$ cells) is filled from a 64-bit seed, so $b = 64$. Our first version extracted only the first byte of each row, a poor choice that failed tests ($Q \approx 40$); using the central column fixes the quality without adding entropy.

3.2.5 Binary digits of π

π is conjectured normal (every binary block occurs with the expected frequency) and passes the tests, but any prefix is computable by a fixed program (we use the Chudnovsky series): its Kolmogorov complexity is $O(\log n)$. The only entropy is the choice of the starting offset: $b = \log_2(N_\pi - 8n) \approx 22.9$ bits for $N_\pi = 8 \cdot 10^6$ bits.

3.3 Physical and manual sources

3.3.1 CPU timing jitter

The delta of a high-resolution clock around a fixed workload varies because of physical effects: oscillator phase noise (thermal and shot noise in the PLL), cache and branch-predictor state, interrupts, bus arbitration. The least significant bit of the delta (after dividing by the clock granularity, which on Windows is 100 ns) carries a small amount of entropy per sample but is *biased and serially correlated*: the raw stream fails the NIST battery ($Q \approx 20$). Conditioning with a hash (SHA-256 over blocks) gives $Q \approx 100$, but the entropy claimed must remain the *measured* min-entropy of the raw stream (Section 4.2), otherwise the hash would manufacture apparent entropy.

3.3.2 Dice and coins

Classical dice and coins are deterministic mechanical systems; unpredictability comes from *sensitivity to initial conditions* amplified by the chaotic bounces, so that a human throw cannot be reproduced. Two caveats:

- **Coins** are not fair in general: Diaconis, Holmes and Montgomery [6] show that a flipped coin lands on the starting side with probability ≈ 0.508 (dynamical bias from precession). A bias p gives $H_\infty = -\log_2 \max(p, 1-p) < 1$ bit per flip.
- **Dice** have manufacturing asymmetries (drilled pips, rounded edges) and human habits (avoiding repeats), lowering H_∞ below $\log_2 6 \approx 2.585$ bits.

For n recorded outcomes of a symbol source we do not assume uniformity; we estimate the min-entropy with the upper confidence bound

$$\hat{p}_u = \min\left\{1, \hat{p}_{\max} + 2.576\sqrt{\frac{\hat{p}_{\max}(1-\hat{p}_{\max})}{n-1}}\right\}, \quad b = n \cdot \min\{\log_2 |\mathcal{A}|, -\log_2 \hat{p}_u\}. \quad (13)$$

For example, 100 rolls of a fair die give $\hat{p}_u \approx 0.26$, i.e. ≈ 1.9 bits/roll rather than 2.585: a small sample cannot certify the nominal entropy, and the honest estimate is lower. To reach 128 bits at the nominal rate one needs $\lceil 128/\log_2 6 \rceil = 50$ rolls; in practice more, and the diagnostic script reports whether the measured value passes 128. The recorded outcomes are compressed to bytes with SHA-256 iterated, which is sound only because b is taken from the measurement, not from the output.

3.3.3 Deck shuffle

A uniform permutation of 52 cards has $\log_2(52!) \approx 225.58$ bits, encoded by the Lehmer code $N = \sum_i c_i(52-i)!$ with $c_i = |\{j > i : \pi_j < \pi_i\}|$ (a bijection onto $\{0, \dots, 52! - 1\}$). Humans do not shuffle uniformly: Bayer and Diaconis [7] show that about seven riffle shuffles are needed for near-uniformity in total variation. The 225.58 bits are an upper bound, not a measurement.

3.3.4 von Neumann extractor and XOR combination

For a coin with $\Pr[H] = p \neq 1/2$ and independent flips, taking disjoint pairs $(H, T) \mapsto 1$, $(T, H) \mapsto 0$, discarding $(H, H), (T, T)$ yields exactly fair bits [8]: both accepted pairs have probability $p(1-p)$. The acceptance rate per pair is $2p(1-p)$, so $\mathbb{E}[\text{flips per bit}] = 1/(p(1-p)) \geq 4$. Independence between flips is essential; correlated flips break the guarantee.

The XOR of independent sources satisfies

$$H_\infty(X_1 \oplus \dots \oplus X_m) \geq \max_j H_\infty(X_j), \quad (14)$$

so combining **secrets** with dice cannot hurt: if the dice are poor, the CSPRNG still gives 256 bits; if the CSPRNG were compromised, the dice still contribute. The bound is a maximum, not a sum, and requires independence.

4 Evaluation Metrics

We use three families of metrics. Each answers a different question and has a blind spot that the others cover.

4.1 NIST SP 800-22 statistical tests

The tests and aggregation criteria follow NIST SP 800-22 [1].

Framework. A test evaluates the null hypothesis H_0 : “the bits are i.i.d. Bernoulli(1/2)” via a statistic with known distribution and returns a p -value. Under H_0 the p -value is uniform on $[0, 1]$, so the test rejects an ideal source with probability $\alpha = 0.01$. *Passing a test is a failure to reject H_0 , not a proof of randomness*; each test only probes one class of alternatives.

Let the bit expansion be $\mathbf{b} \in \{0, 1\}^m$, $m = 8n$, and $X_i = 2b_i - 1$.

Monobit. $s_{\text{obs}} = |\sum_i X_i|/\sqrt{m}$, $p = \text{erfc}(s_{\text{obs}}/\sqrt{2})$. Detects a global bias of ones and zeros.

Runs. With $\pi = \frac{1}{m} \sum_i b_i$ and $V = 1 + \sum_i \mathbf{1}[b_i \neq b_{i+1}]$,

$$p = \operatorname{erfc}\left(\frac{|V - 2m\pi(1 - \pi)|}{2\sqrt{2m}\pi(1 - \pi)}\right), \quad (15)$$

provided $|\pi - \frac{1}{2}| < 2/\sqrt{m}$. Detects too-fast or too-slow oscillation (serial dependence at short range).

Block frequency. For blocks of L bits with ones-proportion π_i , $\chi^2 = 4L \sum_{i=1}^M (\pi_i - \frac{1}{2})^2 \sim \chi_M^2$, $p = \Pr[\chi_M^2 \geq \chi^2]$ (here $L = 128$). Detects local biases that cancel globally.

Cumulative sums (forward and backward). With $S_k = \sum_{i \leq k} X_i$ and $z = \max_k |S_k|$, the p -value follows the distribution of the maximum of a random walk (NIST §2.13), evaluated with the normal CDF Φ over the finite ranges of k . Detects drift of the walk.

Longest run of ones in a block. Blocks of $M = 8$ bits are classified by longest run ($\leq 1, 2, 3, \geq 4$) with null proportions (0.2148, 0.3672, 0.2305, 0.1875), and $\chi^2 = \sum_j (v_j - N\pi_j)^2 / (N\pi_j) \sim \chi_3^2$.

Chi-squared on bytes. With c_v the count of byte value v and $E_v = n/256$, $\chi^2 = \sum_{v=0}^{255} (c_v - E_v)^2 / E_v \sim \chi_{255}^2$. The chi-squared approximation requires $E_v \geq 5$, i.e. $n \geq 1280$; with our $n = 12\,500$, $E_v \approx 48.8$. (The original design used $n = 256$, giving $E_v = 1$, an invalid test; this was corrected.)

4.1.1 Aggregation across samples

One sample produces one p -value per test; a single p -value from one string cannot separate a good generator from a lucky bad one. NIST §4.2 therefore evaluates N independent samples with two criteria per test.

Definition 4.1 (Proportion criterion). The number of passing samples is $\text{Binomial}(N, 1 - \alpha)$ under H_0 ; the source passes if the observed proportion $\hat{\pi} \geq \hat{p}_0 - 3\sqrt{\hat{p}_0(1 - \hat{p}_0)/N}$, with $\hat{p}_0 = 1 - \alpha$. For $N = 100$ the bound is 0.960.

Definition 4.2 (Uniformity criterion). Under H_0 the p -values are uniform. Bin the N p -values in 10 equal intervals with counts F_i and compute $\chi^2 = \sum_{i=1}^{10} (F_i - N/10)^2 / (N/10)$, $P_T = \Pr[\chi_9^2 \geq \chi^2]$. The source passes if $P_T \geq 10^{-4}$ (requires $N \geq 55$).

A test is *OK* only if *both* criteria hold. The second criterion catches sources that pass at $\hat{\pi} = 0.99$ but whose p -values crowd near 0.01–0.05 (a mild but systematic defect). We use $N = 100$ samples of $8n = 10^5$ bits ($n = 12\,500$ bytes).

Limits. With seven tests and two criteria each, an ideal source fails at least one criterion with non-negligible probability. In our runs a **secrets**-like source sometimes has one test out of seven rejected (a $Q \approx 90$ result); this is statistical noise, not a defect, which is why the verdict tolerates a single rejection.

4.2 Min-entropy estimation (NIST SP 800-90B)

The tests above examine the distribution of outputs; they cannot bound the adversary's guessing probability. SP 800-90B [2] estimates min-entropy per symbol of a non-i.i.d. source by running *ten estimators* and taking the *minimum*: each estimator is sound against one kind of structure and blind to the others. On a bit sequence $x_1, \dots, x_L$ the estimators we implement are:

1. Most Common Value (MCV). With $\hat{p}$ the frequency of the most common symbol,

$$p_u = \min \left\{ 1, \hat{p} + 2.576 \sqrt{\frac{\hat{p}(1-\hat{p})}{L-1}} \right\}, \quad H = -\log_2 p_u. \quad (16)$$

It sees only the marginal frequency.

2. Collision. Scan for consecutive collisions. In the binary case the time to the next repeated symbol is $t \in \{2, 3\}$ with $\mathbb{E}[t] = 2 + 2pq$ ($q = 1 - p$). With the sample mean $\bar{t}$, sample standard deviation $\hat{\sigma}$ over v collisions, the lower confidence bound $\bar{t}' = \bar{t} - 2.576\hat{\sigma}/\sqrt{v}$ gives $pq = (\bar{t}' - 2)/2$, $p = \frac{1}{2}(1 + \sqrt{1 - 4pq})$, $H = -\log_2 p$.

3. Markov. Estimate the initial distribution and the 2×2 transition matrix; let $p_{\max}$ be the probability of the most likely 128-bit chain (dynamic programming). Then $H = \min\{1, -\log_2 p_{\max}/128\}$.

4. Compression (Maurer). Partition into $b = 6$ -bit blocks; with D_i the distance to the previous occurrence of block i (a dictionary of $d = 1000$ initial blocks discarded), $\bar{X} = \overline{\log_2 D_i}$, $\hat{\sigma} = 0.5907 \text{sd}(\log_2 D_i)$, $\bar{X}' = \bar{X} - 2.576\hat{\sigma}/\sqrt{v}$. Solve $\mathbb{E}(p) = \bar{X}'$ for p , where

$$\mathbb{E}(p) = G(p) + (2^b - 1)G(q), \quad q = \frac{1-p}{2^b-1}, \quad G(z) = \frac{1}{v} \sum_{t=d+1}^L \left[\sum_{u=1}^{t-1} \log_2 u z^2 (1-z)^{u-1} + \log_2 t z (1-z)^{t-1} \right], \quad (17)$$

and $H = -\log_2 p/b$. It bounds how compressible the source is [11], the information-theoretic view of redundancy.

5. t -Tuple. Let Q_i be the count of the most common i -tuple and t the largest i with $Q_i \geq 35$. With $P_i = Q_i/(L - i + 1)$, $p = \max_{i \leq t} P_i^{1/i}$ and $H = -\log_2 \min\{1, p + 2.576\sqrt{p(1-p)/(L-1)}\}$.

6. Longest Repeated Substring (LRS). With C_j the counts of W -tuples,

$$P_W = \frac{\sum_j \binom{C_j}{2}}{\binom{L-W+1}{2}}, \quad p = \max_{W \geq u} P_W^{1/W}. \quad (18)$$

7–10. Predictors. Four predictors try to guess x_i from the past: MultiMCW (most common value in windows of 63, 255, 1023, 4095), Lag (the value d steps back, $d \leq 128$), MultiMMC (order- k Markov contexts, $k \leq 16$) and LZ78Y (longest matching context up to 16). A scoreboard selects the subpredictor with the highest accumulated success. From the sequence of hits (length N , C correct):

$$p_{\text{global}} = \hat{p} + 2.576 \sqrt{\frac{\hat{p}(1-\hat{p})}{N-1}} \quad (\hat{p} = C/N), \quad H = -\log_2 \max\{p_{\text{global}}, p_{\text{local}}, \frac{1}{2}\}, \quad (19)$$

where p_{local} is the success probability for which a run of length $r+1$ (r the longest observed run of hits) appears in N trials with probability 0.99. We use Feller's approximation: x is the root > 1 of $x = 1 + qp^{r+1}x^{r+2}$ and $\Pr[\text{no run}] = \frac{1-px}{(r+2-(r+1)x)q} x^{-(N+1)}$.

Estimate. $\hat{h} = \min$ of the ten values, in bits per bit; the total is $b = L\hat{h}$.

Deviations from the standard (documented in the code). Tuples are capped at 63 bits; predictors are evaluated on the first 30 000 bits (cost in pure Python); the scoreboard uses cumulative success instead of the standard's short recent window. The standard requires 10^6 bits; with 10^5 the confidence bounds are wider and the estimates are *conservative* (a perfect source typically yields 0.55–0.9 bits/bit; the variation is dominated by the local-run bound).

Why the minimum. The estimators are complementary. In our validation (Section 8) MCV alone gave 0.98 bits/bit on a source with memory whose true min-entropy is 0.32; the collision and predictor estimators detected the memory. Trusting any single estimator would have overstated the entropy by a factor of three.

4.3 Structural tests

Some structures are invisible to the frequency-based tests.

Binary matrix rank. Fill 32×32 matrices over $\mathbb{F}_2$ with consecutive bits. The ranks are 32, 31, ≤ 30 with probabilities 0.2888, 0.5776, 0.1336 (for a uniform random matrix), and $\chi^2 = \sum_j (F_j - N_m p_j)^2 / (N_m p_j)$, $p = e^{-\chi^2/2}$ (2 d.o.f.). Detects linear dependencies between blocks.

Spectral (DFT). The magnitudes $|F_k|$, $k < m/2$, of the discrete Fourier transform of (X_i) should be below $T = \sqrt{\ln(1/0.05)m}$ for 95% of the frequencies. With N_1 the observed count below T ,

$$d = \frac{N_1 - 0.95m/2}{\sqrt{m \cdot 0.95 \cdot 0.05/4}}, \quad p = \operatorname{erfc}(|d|/\sqrt{2}). \quad (20)$$

It detects periodic components.

Approximate entropy. With $\varphi^{(k)} = \sum_i \pi_i \ln \pi_i$ over overlapping k -bit patterns, $\operatorname{ApEn}(k) = \varphi^{(k)} - \varphi^{(k+1)}$ and $\chi^2 = 2m(\ln 2 - \operatorname{ApEn}(k)) \sim \chi_{2k}^2$ ($k = 2$). Detects non-uniformity of short patterns.

Linear complexity. The Berlekamp–Massey algorithm [5] computes the length L of the shortest linear-feedback shift register (LFSR) over $\mathbb{F}_2$ that generates the sequence: at step n the discrepancy is $d_n = s_n \oplus \bigoplus_{i=1}^L c_i s_{n-i}$ and, if $d_n = 1$, the connection polynomial is updated. A random sequence of length n has $L \approx n/2$; an LFSR-generated sequence has L equal to its register length, so $L \ll n/2$ once $n > 2L$. The deviation $\delta = n/2 - L$ has a tail $\Pr \approx 4^{-\delta}$, which we use as a p -value approximation.

Strided linear complexity. A bit-by-bit concatenation of 32-bit words hides the structure of generators whose *word* sequence is linear: their bit stream is a combination of 32 interleaved sequences, so its complexity can reach $32 \cdot 19937 \gg n/2$. Extracting *one bit position of every 32-bit word* (stride 32, $n = 45\,000$ bits) exposes each component sequence with $L = 19937 < n/2 = 22\,500$. This is the only generic test in the battery that separates the Mersenne Twister from a CSPRNG.

4.4 Combining the two axes

Definition 4.3 (Statistical quality). Let F be the fraction of the seven tests (monobit, runs, block frequency, forward and backward cumulative sums, longest run, byte chi-squared) that are OK under both aggregation criteria, and $\hat{H}_\infty$ the mean per-sample MCV min-entropy per byte (with the 99% bound). With H_{ref} the same statistic averaged over 30 `os.urandom` samples of the same length ($H_{\text{ref}} \approx 7.10$ bits/byte at $n = 12\,500$, since the confidence bound is below 8 at finite n),

$$Q = 100 \left(0.7 F + 0.3 \min\{1, \hat{H}_\infty / H_{\text{ref}}\} \right). \quad (21)$$

The reference H_{ref} prevents penalizing an ideal source for the finite sample size.

Definition 4.4 (Verdict).

$$\text{verdict} = \begin{cases} \text{APTO} & Q \geq 85, r \geq 128 \text{ bits}, \\ \text{IMITA AZAR (mimics randomness)} & Q \geq 85, r < 128, \\ \text{FALLA TESTS} & Q < 85. \end{cases} \quad (22)$$

The 128-bit threshold is the security level of secp256k1 (Section 6). The threshold $Q \geq 85$ tolerates *one* rejected test out of seven ($Q = 90$) and rejects two or more ($Q = 80$). It was originally 90, which classified an ideal CSPRNG as failing whenever a single test fell below its bound by chance (this happened in our first complete run: **secrets** obtained $Q = 89.9$). With 7 tests and 2 criteria each, an ideal source rejects at least one criterion with non-negligible probability, so one rejection is not evidence of a defect; two independent rejections are. We deliberately do *not* multiply Q by ad hoc determinism penalties (the first design used factors 0.5 and 0.75): penalties confuse the two concepts. Statistical quality and real entropy are reported separately, and the verdict, not a weighted sum, decides suitability.

4.5 Sources of error in the measurements themselves

The measurement tools can be wrong, and this materially affects conclusions.

- An integer-overflow bug (`2*bits-1` in an unsigned 8-bit type) made `os.urandom` fail the monobit test in 100% of the samples until we validated each test against the NIST SP 800-22 example vectors.
- Two statistics with the same identifier overwrote each other in the aggregation dictionary (forward and backward cumulative sums).
- With $n = 256$ bytes the chi-squared test was invalid ($E_v = 1 < 5$), and mean MCV min-entropy was capped near 5 bits/byte by sample size.
- Reference vectors also validate the cryptographic pipeline (BIP32, BIP39, bech32, descriptor checksums, xpub serialization).

5 Attacks on Generators that Pass the Battery

The following attacks are implemented (`entropy_arena/attacks`) and tested. Each uses only information available to the adversary of Definition 1.1.

State recovery of MT19937. Each output word $y = T(x)$ with T the invertible tempering. From 624 consecutive words ($19968 \geq 19937$ state bits), $x = T^{-1}(y)$ is computed by undoing the shift-and-mask steps (each recovered in ≤ 32 iterations of $x \leftarrow y \oplus (x \ll s \wedge m)$). Loading the recovered state into a clone predicts all future outputs. It recovers the state from 2496 bytes in 0.02 s; the same attack on **secrets** fails, as expected.

State recovery of RANDU. Observing the 8 high bits of each 31-bit state, the unknown 23 low bits of the first state are enumerated (2^{23} candidates); each subsequent observation keeps a candidate with probability 2^{-8} , so ≈ 4 observations leave one. It succeeds from 8 bytes in ≈ 0.3 s.

Offset of the digits of π . The “seed” is an offset in a public constant ($\log_2 \approx 22.9$ bits); a string search of 32 observed bytes locates it.

Clock-seeded generators. If the generator is seeded with `int(time.time())`, the search space for a day is 86 400 seeds; brute force finds the seed in at most ≈ 0.6 s.

Linear complexity (Section 4) is the statistical counterpart of the MT attack: $2L = 39\,874$ bits of a single bit position determine the recurrence.

These results give the operational meaning of the r axis: $r < 128$ is not an abstract number, it is the distance to an attack that runs in seconds.

6 Cryptographic Pipeline

6.1 From entropy to seed (BIP39)

256 bits of entropy E are encoded as a 24-word mnemonic $\mathbf{m}$ (the entropy plus an 8-bit SHA-256 checksum, in groups of 11 bits indexing a 2048-word list). The seed is derived from the *mnemonic sentence*, not from the raw entropy:

$$S = \text{PBKDF2-HMAC-SHA512}(\mathbf{m}, \text{"mnemonic"} \| P, 2048, 64). \quad (23)$$

(The first prototype derived it from the raw bytes, a deviation from the standard that would make wallets incompatible with any other software.) The generator *refuses* to build a signer if the source reports $r < 128$ bits.

6.2 Hierarchical derivation (BIP32)

With $I = \text{HMAC-SHA512}(\text{"Bitcoin seed"}, S)$, the master key is $(k_m, c_m) = (I_L, I_R)$. The child i of (k, c) is

$$I = \text{HMAC}(c, \underbrace{0x00 \| k}_{i \geq 2^{31}} \text{ or } \underbrace{\text{ser}_P(kG)}_{i < 2^{31}} \| i), \quad k_i = (I_L + k) \bmod n_G, \quad c_i = I_R, \quad (24)$$

where G is the `secp256k1` base point and n_G its order. The *public* derivation (CKDpub) computes $K_i = I_L G + K$ from the parent public key only and is defined solely for non-hardened indices. This is the mathematical basis of watch-only wallets: from an extended public key one derives all receiving addresses but no private key; hardened steps stop an extended public key from revealing sibling private keys.

The account path is BIP48, $m/48'/c'/0'/2'$ ($c = 0$ mainnet, $c = 1$ testnet/regtest; script type 2 = P2WSH). Extended keys are serialized (Base58Check) as

$$\text{version} \| \text{depth} \| \text{parent fp.} \| \text{index} \| \text{chain code} \| \text{key}, \quad (25)$$

where the *fingerprint* is the first four bytes of `RIPEMD160(SHA256(K))` of the master public key.

Curve arithmetic backend. Point multiplication (kG) and point addition ($I_L G + K$ for CKDpub) run through `coincurve` (Python bindings to `libsecp256k1`, the constant-time C library used by Bitcoin Core) when it is installed; otherwise the code falls back to a pure-Python implementation (`secp256k1.py`) kept for portability and cross-checking. The two backends are verified against each other in the test suite (identical public keys and CKDpub outputs for the same inputs) and against BIP32 test vector 1. The pure-Python path remains non-constant-time and unaudited; `coincurve` materially improves reliability but does not, by itself, make the surrounding prototype (key storage, PSBT handling, this codebase's own serialization) fit for real funds.

6.3 Multisignature script and descriptors

For threshold t and public keys $K_1, \dots, K_n$ sorted lexicographically (BIP67), the witness script is

$$\text{OP_t } \langle K_{(1)} \rangle \cdots \langle K_{(n)} \rangle \text{ OP_n OP_CHECKMULTISIG}, \quad (26)$$

and the output is the P2WSH program $\text{SHA256}(\text{script})$, encoded in bech32 (BIP173, a BCH code with polymod checksum). The script accepts a transaction T iff there exists $\mathcal{S} \subseteq \{1, \dots, n\}$, $|\mathcal{S}| \geq t$, with $\text{Verify}(K_i, T, \sigma_i) = 1$ for $i \in \mathcal{S}$. The public wallet configuration is the *output descriptor*

$$\text{wsh}(\text{sortedmulti}(2, [\text{fp}_1/48\text{h}/1\text{h}/0\text{h}/2\text{h}] \text{tpub}_1 / <0;1> / *, \dots)) \# c,$$

whose 8-character checksum c is a BCH code (BIP380) that detects transcription errors. It contains fingerprints, origin paths, extended public keys and policy, and none of the private material.

6.4 Spending logic: the policy as a Boolean formula

Let $\sigma_A, \sigma_B, \sigma_C \in \{0, 1\}$ indicate whether a valid signature of the corresponding key on the transaction T is supplied. The 2-of-3 policy encoded in the descriptor is the Boolean formula

$$\text{Spend}(T) = (\sigma_A \wedge \sigma_B) \vee (\sigma_A \wedge \sigma_C) \vee (\sigma_B \wedge \sigma_C). \quad (27)$$

If *any* parenthesis evaluates to 1 the output can be spent; each parenthesis is a *minimal authorized set*, and no single key is one. For a general t -of- n policy the formula is the disjunction, over the $\binom{n}{t}$ subsets of size t , of the conjunction of the signatures in the subset (3 clauses for 2-of-3, 10 for 3-of-5).

Proposition 6.1 (DNF = threshold). *Equation (27) holds if and only if $\sigma_A + \sigma_B + \sigma_C \geq 2$.*

Proof. If some pair has both signatures, the sum is at least 2. Conversely, a sum of at least 2 among three binary values means at least two of them equal 1, and those two form one of the three clauses. $\square$

The threshold form $\sum_i \sigma_i \geq t$ is what `OP_CHECKMULTISIG` evaluates, and it is why the descriptor uses the compact `sortedmulti(2,  $K_A, K_B, K_C$ )`: a naive translation of Eq. (27) into script (`or(and(pk(A),pk(B)),...)`) is logically equivalent but repeats every key twice (6 key slots instead of 3), enlarging the witness script and therefore the fees. The two forms accept exactly the same transactions.

6.5 Generalization: t -of- n and the combinatorics of the “10 favorable cases”

Equation (27) and Proposition 6.1 generalize directly.

Definition 6.2 (Threshold spending rule). Let $\sigma_1, \dots, \sigma_n \in \{0, 1\}$ be indicator variables, $\sigma_i = \mathbf{1}[\text{Verify}(K_i, T, \sigma_i^{\text{ECDSA}}) = 1]$, one per signer. A t -of- n policy defines

$$\text{Spend}(T) = \mathbf{1} \left[\sum_{i=1}^n \sigma_i \geq t \right]. \quad (28)$$

Proposition 6.3 (Threshold form $\equiv$ disjunctive normal form). *Let $\binom{[n]}{t} = \{\mathcal{S} \subseteq \{1, \dots, n\} : |\mathcal{S}| = t\}$ be the set of t -element subsets of signers. Then Eq. (28) is logically equivalent to*

$$\text{Spend}(T) = \bigvee_{\mathcal{S} \in \binom{[n]}{t}} \bigwedge_{i \in \mathcal{S}} \sigma_i, \quad (29)$$

a disjunction of $\binom{n}{t}$ clauses, each the conjunction of exactly t signatures (a minimal authorized set: removing one signer from $\mathcal{S}$ makes the clause unsatisfiable).

Proof. ($\Leftarrow$) If some clause $\bigwedge_{i \in \mathcal{S}} \sigma_i = 1$ for $|\mathcal{S}| = t$, then $\sigma_i = 1$ for all $i \in \mathcal{S}$, so $\sum_i \sigma_i \geq t$. ($\Rightarrow$) If $\sum_i \sigma_i \geq t$, let $\mathcal{S}$ be any t -element subset of $\{i : \sigma_i = 1\}$ (which has at least t elements); its clause is satisfied. Minimality holds because dropping one index from $\mathcal{S}$ leaves $t - 1 < t$ signatures, which by Definition 6.2 does not satisfy the policy on its own. $\square$

Corollary 6.4 (Why the count is $\binom{n}{t}$). *The number of favorable (minimal, authorized) signer sets is*

$$\binom{n}{t} = \frac{n!}{t!(n-t)!} = \frac{n(n-1) \cdots (n-t+1)}{t!}. \quad (30)$$

This counts unordered selections of t signers out of n : order does not matter because `OP_CHECKMULTISIG` (and the attack model of Section 7) only cares about which keys sign, not the order in which signatures are supplied or verified.

The 2-of-3 case (Standard track). $\binom{3}{2} = \frac{3 \cdot 2}{2!} = 3$ clauses, exactly the three rows of Table 1 with `Spend` = 1 and exactly two ones.

The 3-of-5 case (Advanced track): why 10. With signers $\{A, B, C, D, E\}$ and $t = 3$, $n = 5$,

$$\binom{5}{3} = \frac{5!}{3!2!} = \frac{5 \cdot 4 \cdot 3}{3 \cdot 2 \cdot 1} = \frac{60}{6} = 10. \quad (31)$$

Two equivalent ways to see the 10: (i) *choose the 3 signers who sign*: 5 choices for the first, 4 for the second, 3 for the third, $5 \cdot 4 \cdot 3 = 60$ *ordered* triples, divided by $3! = 6$ orderings of the same triple, giving $60/6 = 10$; (ii) *choose the 2 signers who do **not** sign* instead, since $\binom{5}{3} = \binom{5}{2} = \frac{5 \cdot 4}{2} = 10$ by the symmetry $\binom{n}{t} = \binom{n}{n-t}$ — fixing which 3 of 5 sign is the same combinatorial object as fixing which 2 of 5 stay out. The 10 minimal authorized sets are

$$\begin{aligned} &\{A, B, C\}, \{A, B, D\}, \{A, B, E\}, \{A, C, D\}, \{A, C, E\}, \\ &\{A, D, E\}, \{B, C, D\}, \{B, C, E\}, \{B, D, E\}, \{C, D, E\}, \end{aligned} \quad (32)$$

so that, generalizing Eq. (27),

$$\text{Spend}_{3\text{-of-5}}(T) = (\sigma_A \sigma_B \sigma_C) \vee (\sigma_A \sigma_B \sigma_D) \vee \cdots \vee (\sigma_C \sigma_D \sigma_E) \quad (10 \text{ clauses}), \quad (33)$$

writing $\wedge$ as multiplication. As with 2-of-3, the on-chain script never expands this DNF: `sortedmulti(3,  $K_A, \dots, K_E$ )` encodes Eq. (28) directly (5 key slots, one `OP_CHECKMULTISIG`); Bitcoin’s stack machine checks the sum of verified signatures against t in a single pass, so the 10-clause form describes *which* signer sets the script accepts, not how it is executed. (Bonus recognition, per the assignment rubric, requires all four students to control one signer directly and the fifth key to have a documented recovery/offline role; see Section 7 for why 3-of-5 earns it.)

Table 1: Truth table of the 2-of-3 policy (1 = valid signature supplied).

σ_A	σ_B	σ_C	Spend	Clause satisfied
0	0	0	0	—
1	0	0	0	none (a key alone is not enough)
0	1	0	0	none
0	0	1	0	none
1	1	0	1	$\sigma_A \wedge \sigma_B$
1	0	1	1	$\sigma_A \wedge \sigma_C$
0	1	1	1	$\sigma_B \wedge \sigma_C$
1	1	1	1	all three (redundant)

What this answers. The formula and its truth table answer two of the three questions a reviewer can ask:

- (1) *What exact condition protects the funds?* Eq. (27): at least two of the three keys must sign the same transaction.
- (2) *What happens if signers are lost or compromised?* Read it off Table 1. Let ℓ be the number of keys lost and c the number an adversary controls. The legitimate owners can still spend iff $n - \ell \geq t$, and the adversary can spend iff $c \geq t$. For 2-of-3: losing one key leaves exactly one clause satisfiable by the two remaining keys (no single point of failure; migrate to a fresh wallet); losing two satisfies no clause and the funds are locked; one compromised key satisfies no clause on its own; two compromised keys satisfy one clause and the funds can be stolen.

The third question (what public information is enough to reconstruct the wallet without being able to spend) is answered by the descriptor itself: it fixes the policy t , the origin paths, the fingerprints and the extended public keys, so CKDpub yields every address and witness script, but it contains no σ_i -producing material (Section 6).

Empirical check. The `regtest` end-to-end run (`experiments/regtest_e2e.py`) exercises every row of Table 1 that matters against Bitcoin Core: each key alone fails to finalize the PSBT (3 rows), and each of the three pairs ($A+B$, $A+C$, $B+C$) finalizes and yields a valid transaction whose witness is $[\emptyset, \sigma_i, \sigma_j, \text{witnessScript}]$.

6.6 Verification chain

Every layer is validated against independent implementations: BIP32 test vector 1 (xpub, xprv, fingerprint), BIP39 (TREZOR vector), BIP173, the BIP380 checksum vector; and end-to-end against Bitcoin Core in `regtest`: three signers with separate wallets (each holding only its own `tprv`), a watch-only coordinator whose address coincides with our public derivation, a funded PSBT signed independently by two signers, a rejected finalization with one signature, and the confirmed transaction whose witness is $[\emptyset, \sigma_a, \sigma_b, \text{witnessScript}]$.

7 Security Composition in Threshold Schemes

Let the n signers use independent keys with min-entropies $h_1 \leq h_2 \leq \dots \leq h_n$ (bits). A key with h bits of entropy costs the adversary $\approx 2^h$ trials; but a public key also carries an ECDLP barrier of $\approx 2^{128}$ operations, so the effective strength of one key is $\min\{h_i, 128\}$.

Proposition 7.1 (*t-of-n security*). *To spend, the adversary must recover t keys. Attacking the t weakest keys, the expected work is $\sum_{i \leq t} 2^{\min(h_i, 128)} \approx 2^{\min(h_{(t)}, 128)}$, where $h_{(t)} = h_t$ is the t -th smallest min-entropy. The wallet's security (in bits) is therefore $\min\{h_{(t)}, 128\}$.*

Corollary 7.2 (Correction of a common belief). *The security of a t-of-n scheme is not that of the strongest source. For 2-of-3, a single strong key is not enough: if two sources are weak, the adversary breaks both and spends without the strong key. To have s-bit security we need at least $n - t + 1$ strong keys ($= 2$ for 2-of-3). Equivalently: the design tolerates the compromise of up to $t - 1$ keys after generation (theft, loss), but requires that at least $n - t + 1$ keys be generated with strong entropy.*

The earlier version of this document claimed $P_{\text{guess}} \leq 2^{-H_\infty^*}$ if a single source is honest; that is true for an n -of- n (unanimity) scheme, where the adversary needs *every* key, not for t -of- n . The code implements Proposition 7.1 as the “attack effort” reported by the vault builder.

Loss and compromise model. With a 2-of-3 policy: losing one key leaves the funds spendable by the other two (a migration is advisable); losing two locks the funds; compromising one key does not allow spending; compromising two does. Independence of the signers (separate machines, separate entropy sources) is a requirement of the argument, not just of the procedure.

7.1 Why 3-of-5 is more secure than 2-of-3

Proposition 7.1 already gives one reason: per-key entropy being equal, security is $\min\{h_{(t)}, 128\}$, so a scheme with $t = 3$ throws away the two *weakest* of five keys before the entropy floor is even evaluated, whereas $t = 2$ throws away only one of three. We now formalize a second, independent reason that holds even when every individual key has *identical* strength: raising t makes the attacker's job combinatorially, not just additively, harder.

Definition 7.3 (Independent compromise model). Suppose each signer's key is independently compromised (malware, coercion, device theft) with probability $p \in (0, 1)$, identical across signers. Let $X \sim \text{Binomial}(n, p)$ be the number of compromised keys. The wallet is broken iff $X \geq t$ (Definition 6.2 applied to the adversary).

Proposition 7.4 (Breaking probability and its leading order).

$$P_{\text{break}}(t, n, p) = \Pr[X \geq t] = \sum_{k=t}^n \binom{n}{k} p^k (1-p)^{n-k}. \quad (34)$$

As $p \rightarrow 0$,

$$P_{\text{break}}(t, n, p) = \binom{n}{t} p^t + O(p^{t+1}), \quad (35)$$

i.e. the leading behavior is governed by $\binom{n}{t} p^t$: the exponent t (not n) is what dominates for small, realistic compromise probabilities.

Proof. Every term with $k > t$ in the sum contributes $O(p^{t+1})$ or smaller since $k \geq t+1$; only the $k = t$ term is $O(p^t)$, and $(1-p)^{n-t} = 1 + O(p)$, giving $\binom{n}{t} p^t (1 + O(p)) = \binom{n}{t} p^t + O(p^{t+1})$. $\square$

Corollary 7.5 (2-of-3 vs. 3-of-5).

$$P_{\text{break}}(2, 3, p) = 3p^2(1-p) + p^3 = 3p^2 - 2p^3 = 3p^2 + O(p^3), \quad (36)$$

$$P_{\text{break}}(3, 5, p) = 10p^3(1-p)^2 + 5p^4(1-p) + p^5 = 10p^3 - 15p^4 + 6p^5 = 10p^3 + O(p^4), \quad (37)$$

using $\binom{3}{2} = 3$ and $\binom{5}{3} = 10$ from Corollary 6.4. Hence

$$\frac{P_{\text{break}}(3, 5, p)}{P_{\text{break}}(2, 3, p)} \xrightarrow{p \rightarrow 0} \frac{10p^3}{3p^2} = \frac{10}{3} p \rightarrow 0, \quad (38)$$

so for any compromise probability small enough to be operationally realistic ($p \ll 1$, e.g. an individually well-secured hardware signer), 3-of-5 is strictly and increasingly safer than 2-of-3 as p decreases — even though it exposes more keys ($5 > 3$) to the adversary. Concretely, at $p = 0.05$: $P_{\text{break}}(2, 3, 0.05) \approx 7.25 \times 10^{-3}$ while $P_{\text{break}}(3, 5, 0.05) \approx 1.16 \times 10^{-3}$, a 2-of-3 wallet is about $6.3\times$ more likely to be broken than a 3-of-5 wallet under the same per-key compromise risk.

Remark 7.6 (Why more clauses does not mean more attack surface). It may seem that $\binom{5}{3} = 10 > \binom{3}{2} = 3$ authorized sets gives the adversary *more* ways in. It does not: each clause requires $t = 3$ simultaneous compromises instead of $t = 2$, and Eq. (35) shows the extra clause *count* (10 vs. 3, a factor $3.3\times$) is dominated by the extra power of p (a factor p , which is $\ll 1$ by assumption). The count $\binom{n}{t}$ sets the *prefactor*; the exponent t sets the *order of magnitude*.

Equating the two leading terms, $3p^2 = 10p^3$, gives the exact crossover $p^* = 3/10 = 0.3$: for every $p < 0.3$ — any operationally realistic compromise probability — $3p^2 > 10p^3$ and 3-of-5’s leading-order breaking probability is strictly smaller, as the numeric example above ($p = 0.05$) confirms.

Proposition 7.7 (Loss tolerance). *The scheme tolerates the loss (not compromise) of up to $n - t$ keys before funds become unspendable. For 2-of-3, $n - t = 1$; for 3-of-5, $n - t = 2$.*

Proof. Funds remain spendable iff at least t keys survive, i.e. iff the number lost is at most $n - t$. $\square$

Policy	Compromises needed to steal	Losses tolerated	P_{break} leading term
2-of-3	2	1	$3p^2$
3-of-5	3	2	$10p^3$

In summary, 3-of-5 dominates 2-of-3 on *both* axes at once — it needs one more simultaneous compromise to steal (Corollary 7.5) *and* tolerates one more loss before funds are frozen (Proposition 7.7) — at the operational cost of coordinating two more signers and requiring three, not two, of them for every legitimate spend. This is exactly the trade-off the assignment’s bonus rubric rewards: $n - t + 1 = 3$ people must independently generate strong-entropy keys (Section 6 rejects any source below 128 bits), one more than the 2 required by 2-of-3.

8 Results

All numbers come from the code in this repository. Arena: $N = 100$ samples of $n = 12\,500$ bytes (10^5 bits) per method; r is bits per sample, capped at $8n = 10^5$ for CSPRNG sources.

8.1 Arena

Method	Q	r (bits)	Verdict
<code>os.urandom</code>	100	10^5 (cap)	APTO
<code>secrets</code>	89.9	10^5 (cap)	APTO
XOR(MT seed 42, <code>os.urandom</code>)	100	10^5 (cap)	APTO
CPU jitter + SHA-256	100	1.24×10^4	APTO
XOR(MT, Brownian, jitter+SHA-256)	99.9	1.32×10^4	APTO
NumPy PCG64 (128-bit seed)	100	128	APTO
Mersenne Twister (unseeded)	100	0	mimics randomness
Mersenne Twister (seed 42)	90.0	0	mimics randomness
Brownian motion (simulated)	99.9	64	mimics randomness
Rule 30 (central column)	100	64	mimics randomness
Logistic map ($r = 3.99$, low bit)	100	64	mimics randomness
Digits of π	100	22.9	mimics randomness
RANDU	100	30	mimics randomness
UUIDv4	37.5	95,312	fails tests
CPU jitter, raw	19.8	1.17×10^4	fails tests
Lorenz attractor	15.1	64	fails tests

Table 2: Arena results, $N = 100$ samples of 10^5 bits per method (file `results/leaderboard.json`). The values $Q = 89.9$ (`secrets`) and $Q = 90.0$ (MT seed 42) each correspond to *one* of seven tests rejected by chance; the verdict tolerates a single rejection.

Reading the table. Almost every method reaches $Q \geq 99$, but only those with a fresh unpredictable input of at least 128 bits are suitable: `os.urandom/secrets`, PCG64 (seeded per sample with 128 bits), the XOR with a CSPRNG, and the hashed jitter, whose $r \approx 1.2 \times 10^4$ bits per 10^5 raw bits is the SP 800-90B minimum over the ten estimators on the *raw* stream (a conservative estimate: the hash is credited only with what was measured). The XOR of MT, Brownian motion and jitter is APTO *only because* of the jitter component, consistent with $H_\infty(\oplus X_j) \geq \max_j H_\infty(X_j)$. The Mersenne Twister, Brownian motion, Rule 30, π , RANDU and the logistic map are indistinguishable from a uniform source by the NIST battery and have a min-entropy per byte of ≈ 7.10 (the reference), yet Theorem 2.5 caps their entropy by 0–64 bits. In the quality/entropy plane they occupy the corner of *high Q*, *low r*, which is exactly what an attacker exploits. Conversely UUIDv4 has abundant entropy (95,312 bits) but poor Q because six constant bits per 16 bytes leave a signature: the two axes are independent in both directions. Lorenz fails for a reason already computed: it is oversampled by a factor of ≈ 75 .

8.2 Validation of the min-entropy estimators

Source	True H_∞ /bit	MCV alone	Min of 10 estimators
Periodic (0110 repeated)	0	0.99	0
Biased ($\Pr[1] = 0.7$)	0.515	0.51	0.32
Sticky ($\Pr[\text{repeat}] = 0.8$)	0.322	0.98	0.17
<code>os.urandom</code>	1	0.98	0.62

Table 3: Estimators on sources of known min-entropy (10^5 bits; `experiments/validate_estimators.py`, fixed seed). No estimate exceeded the true value. The estimator that sets the minimum is collision (periodic, sticky), compression (biased) and MultiMMC (`os.urandom`).

MCV alone overestimates the memoryful sources by up to $3\times$; taking the minimum never overestimates, at the price of underestimating ideal sources by $\approx 40\%$ at this sample size.

8.3 Structural tests

Method	Rank	DFT	ApEn	Lin. compl.	Strided LC
<code>secrets</code>	90%	100%	90%	100%	100%
PCG64	100%	100%	100%	100%	90%
Mersenne Twister	100%	100%	100%	100%	0%
RANDU	100%	100%	100%	100%	100%
Digits of π	100%	90%	100%	100%	100%
Rule 30	100%	100%	100%	100%	–
Logistic map	100%	100%	100%	100%	–

Table 4: Proportion of 10 samples (65 536 bits; 180 kB for strided LC) passing each test. A 90% for an ideal source (`secrets`, PCG64) is one rejection in ten at $\alpha = 0.01$: statistical noise, not a defect. Only the Mersenne Twister row reaches 0%.

Only the strided linear complexity separates the Mersenne Twister ($L = 19\,937$ vs. $22\,500$ expected; 0% of samples pass). RANDU passes because it outputs the high byte of a *nonlinear-over- $\mathbb{F}_2$* congruential state, so its weakness is arithmetic (the planes) rather than $\mathbb{F}_2$ -linear. This is the general pattern: *every test is a filter for one algebraic structure*; the absence of a detected structure says nothing about undetected ones.

8.4 Attacks

Attack	Observed data	Time	Success
MT19937 state recovery	2 496 B	0.02 s	yes
RANDU state recovery	8 B	0.26 s	yes
Offset of π (search; π precomputed)	32 B	0.08 s	yes
Clock seed (1 day)	86 400 candidates	≤ 0.6 s [†]	yes
Same MT attack on secrets	2 496 B	–	no

Table 5: Attacks against sources with $Q \geq 85$ (**results/attacks.txt**). [†]Time depends on the position of the secret seed within the day (measured 0.07–0.54 s).

8.5 Wallet

The regtest test (2-of-3 P2WSH) completes all its checks: separate wallets with one private key each, watch-only address identical to Core’s, one signature not sufficient to finalize, two sufficient (any pair), transaction confirmed with witness $[\emptyset, \sigma, \sigma, \text{script}]$. The generated public descriptor is accepted by Core with the same checksum.

9 Discussion and Limitations

Why imitating randomness does not make a model good. Statistical tests measure a property of the *distribution of outputs* (D looks like U_n to the tested statistics); cryptographic security is the *adversary’s guessing probability given what he knows*, i.e. conditional min-entropy. For a deterministic g with a seed of s bits, Theorem 2.5 bounds the latter by s whatever the former. The two are decoupled in both directions (UUIDv4 fails tests with high entropy; the Mersenne Twister passes tests with none), so a single score conflates two questions.

Why the physical sources work. A physical source injects entropy that is not a function of a bounded seed: thermal noise, timing jitter, chaotic mechanical amplification. Its difficulties are not statistical but *metrological*: bias (coin 0.508), serial correlation (jitter, dice habits), and, above all, the difficulty of *certifying* the entropy rate. Hence the two-step design: measure a conservative min-entropy of the raw outcomes, then condition with a hash for uniform bytes, never taking the hash output’s statistics as evidence of entropy.

Why simulated physics does not work. Brownian motion and the Lorenz flow are physical models, but a computer executes a deterministic approximation with a bounded seed. Entropy enters only through the seed; the model can amplify it (Lorenz, at h_{KS} bits per unit time) but not create it, and oversampling beyond h_{KS} reads the same information repeatedly.

Limitations.

- The estimators are adapted, not certified, and use 10^5 bits (the standard asks 10^6); estimates are conservative.
- The battery is a subset of NIST SP 800-22 plus four structural tests; it does not include Diehard/TestU01/PractRand.
- A manual session is one fixed sample: uniformity of p -values does not apply and the NIST proportions are unreliable (the arena warns).
- Humans do not shuffle or flip uniformly; the deck bits are an upper bound, and dice/coin bits are measured lower bounds at 99% confidence.

- Elliptic-curve arithmetic is pure Python and not side-channel resistant: the pipeline is intended for regtest/testnet only.
- An ideal source may fail one test out of seven by chance ($\sim 1\text{--}2\%$ per criterion).

Conclusions. (i) Q and r must be reported separately. (ii) The only sources that supply key entropy are those with a fresh unpredictable input of at least 128 bits: a CSPRNG (OS entropy pool), or a physically measured source whose min-entropy has been certified; mathematical maps, Mersenne Twister, RANDU, π and cellular automata are deterministic and only propagate their seed. (iii) A t -of- n wallet is as strong as the t -th weakest key. (iv) Every measurement tool must be validated against known vectors: several of the phenomena reported by the arena were, at first, defects of the arena.

A Notation

Symbol	Meaning
H_1, H_2, H_∞	Shannon, collision, and min-entropy
$H_\infty(D Z)$	Conditional min-entropy given adversary knowledge Z
$n, m = 8n$	Bytes and bits per sample
N	Number of samples per method (100)
α	Significance level (0.01)
Q	Statistical quality, 0–100
r	Real entropy (bits per sample)
$b(M)$	Bits of fresh unpredictable input consumed
h_{KS}	Kolmogorov–Sinai entropy rate
λ	Lyapunov exponent
k_m, c_m	BIP32 master private key and chain code
t, n	Signature threshold, total number of signers (Section 7)
$h_{(t)}$	t -th smallest min-entropy among the n keys
σ_i	Indicator: valid signature of signer i on T supplied
$\binom{n}{t}$	Number of minimal t -of- n authorized signer sets
p	Per-key independent compromise probability
$P_{\text{break}}(t, n, p)$	Probability an adversary controls $\geq t$ of n keys

B Constants

Quantity	Value
$\log_2 6$	2.58496...
$\log_2(52!)$	225.581...
MT19937 period	$2^{19937} - 1$
MT19937 state and word count	19937 bits, 624 words
secp256k1 group order	$\approx 1.1579 \times 10^{77}$
ECDLP work (Pollard rho)	$\approx 2^{128}$
PBKDF2 iterations (BIP39)	2048
Lorenz largest Lyapunov exponent	≈ 0.9056
Logistic map Lyapunov at $r = 4$	$\ln 2$
Coin: probability of landing on the starting side	≈ 0.508

References

- [1] L. E. Bassham III et al., *A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications*, NIST SP 800-22 Rev. 1a, 2010.

- [2] M. S. Turan et al., *Recommendation for the Entropy Sources Used for Random Bit Generation*, NIST SP 800-90B, 2018.
- [3] M. Matsumoto and T. Nishimura, Mersenne Twister: a 623-dimensionally equidistributed uniform pseudo-random number generator, *ACM Trans. Model. Comput. Simul.* 8(1), 1998.
- [4] G. Marsaglia, Random numbers fall mainly in the planes, *Proc. Natl. Acad. Sci. USA* 61(1), 1968.
- [5] J. L. Massey, Shift-register synthesis and BCH decoding, *IEEE Trans. Inf. Theory* 15(1), 1969.
- [6] P. Diaconis, S. Holmes and R. Montgomery, Dynamical bias in the coin toss, *SIAM Review* 49(2), 2007.
- [7] D. Bayer and P. Diaconis, Trailing the dovetail shuffle to its lair, *Ann. Appl. Probab.* 2(2), 1992.
- [8] J. von Neumann, Various techniques used in connection with random digits, *Applied Math. Series* 12, National Bureau of Standards, 1951.
- [9] E. N. Lorenz, Deterministic nonperiodic flow, *J. Atmos. Sci.* 20(2), 1963.
- [10] S. Wolfram, Random sequence generation by cellular automata, *Adv. Appl. Math.* 7, 1986.
- [11] U. M. Maurer, A universal statistical test for random bit generators, *J. Cryptology* 5(2), 1992.
- [12] Bitcoin Improvement Proposals BIP32, BIP39, BIP48, BIP67, BIP173, BIP380, <https://github.com/bitcoin/bips>.